

TéléPro CI

Guide de déploiement en production

VPS · PostgreSQL · Node.js · Nginx · PM2 · SSL

Ce document couvre

1. Configuration de l'environnement de développement

2. Installation et configuration de l'application

Généré le 20 avril 2026

1. Peut-on utiliser O2switch ?

Analyse de compatibilité

O2switch est un hébergement mutualisé français très populaire (offre unique ~7 €/mois). Malheureusement, il manque plusieurs prérequis techniques de TéléPro CI.

Compatibilité O2switch vs besoins TéléPro CI

Fonctionnalité	O2switch mutualisé	Besoin TéléPro CI
PostgreSQL 16	' MySQL uniquement	' Obligatoire (driver pg)
Node.js autonome	Limité (Passenger)	' PM2 + ports libres
Accès root SSH	'	' Pour PM2 et services
PM2	'	' Redémarrage auto
Config Nginx custom	'	' Proxy API + gzip
Docker	'	Optionnel
Ports personnalisés	'	' Port 3001 (API)
Variables env système	Limité (cPanel)	' JWT_SECRET, DATABASE_URL

& Blocage principal : O2switch ne propose que MySQL/MariaDB. Le projet interdit explicitement la migration vers MySQL (driver pg, syntaxe \$1/\$2, RETURNING *, triggers PL/pgSQL). Migrer la DB représenterait plusieurs semaines de travail.

Cas où O2switch reste utile

- Gestionnaire de nom de domaine et DNS (pointer vers votre VPS)
- Héberger le frontend uniquement (upload du dossier dist/ par FTP)
- Adresse email professionnelle @votre-domaine.ci

Comparaison des hébergeurs VPS recommandés

Hébergeur	Offre	RAM/CPU	Prix/mois	Latence CI
Contabo &	VPS S	8 Go / 4 vCPU	~6 €	Bon
Hetzner	CX22	4 Go / 2 vCPU	~4 €	Bon
OVH	VPS Starter	2 Go / 1 vCPU	~6 €	Bon (Paris)
DigitalOcean	Droplet Basic	4 Go / 2 vCPU	~24 \$	Correct
Scaleway	DEV1-S	2 Go / 2 vCPU	~4 €	Bon (Paris)

3 scénarios possibles

- **Contrôle total** : PostgreSQL, PM2, Nginx, SSL sur un seul serveur
- **Coût** : 4-8 €/mois • **Compétence requise** : administration Linux basique
- Idéal si vous voulez héberger plusieurs instances (multi-clients)

- **Zéro administration serveur** : Railway gère Node.js, Neon gère PostgreSQL
- **Coût** : ~10 \$/mois • **Déploiement** en 30 min depuis GitHub
- Idéal si vous ne voulez pas gérer l'infra

- **Garder O2switch** pour le frontend statique + domaine
- **Petit VPS Hetzner** (~4 €) pour l'API Express + PostgreSQL
- Idéal si vous avez déjà un abonnement O2switch actif

2. Option A — Installation complète sur VPS

Ubuntu 22.04 LTS · Node.js 20 · PostgreSQL 16 · Nginx · PM2

Étape 1 — Sécurisation initiale du serveur

bash — sécurisation

```
# Connexion SSH initiale
ssh root@IP_DU_SERVEUR

# Créer un utilisateur dédié
adduser telepro
usermod -aG sudo telepro
rsync --archive --chown=telepro:telepro ~/.ssh /home/telepro

# Pare-feu (ouvrir uniquement SSH, HTTP, HTTPS)
ufw allow OpenSSH && ufw allow 80 && ufw allow 443 && ufw enable

# Se reconnecter en tant qu'utilisateur non-root
ssh telepro@IP_DU_SERVEUR
```

Étape 2 — Installer les dépendances

bash — installation

```
# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# PostgreSQL 16
sudo apt install -y postgresql postgresql-contrib

# Nginx + Certbot (SSL)
sudo apt install -y nginx certbot python3-certbot-nginx

# PM2 (gestionnaire de processus Node.js)
sudo npm install -g pm2

# Vérifications
node -v          # !' v20.x
psql --version   # !' PostgreSQL 16.x
nginx -v         # !' nginx/1.24.x
pm2 -v           # !' 5.x
```

Étape 3 — Configurer PostgreSQL

psql — configuration base de données

```
sudo -u postgres psql

-- Créer l'utilisateur et la base
CREATE USER telepro WITH PASSWORD 'VotreMotDePasseForte!';
CREATE DATABASE telepro_ci OWNER telepro;
GRANT ALL PRIVILEGES ON DATABASE telepro_ci TO telepro;
\q

# Tester la connexion
psql postgresql://telepro:VotreMotDePasseForte!@localhost:5432/telepro_ci
```

Étape 4 — Déployer le code

bash — déploiement du code

Option Git (recommandé)

cd /home/telepro

git clone https://github.com/VOTRE_ORG/telepro-standard.git app

cd app

OU rsync depuis votre machine locale

rsync -avz --exclude node_modules --exclude .git \

./ telepro@IP_DU_SERVEUR:/home/telepro/app/

2. Option A — Installation VPS (suite)

Étape 5 — Fichier d'environnement

```
bash
```

```
cd /home/telepro/app/apps/api
cp .env.example .env
nano .env
```

```
apps/api/.env
```

```
NODE_ENV=production
PORT=3001

# Connexion PostgreSQL
DATABASE_URL=postgresql://telepro:VotreMotDePasseForte!@localhost:5432/telepro_ci

# Générer avec : npm run gen-secret (min 64 caractères)
JWT_SECRET=VOTRE_SECRET_64_CHARS_GENERE_AVEC_NPM_RUN_GEN_SECRET

# Twilio SMS/WhatsApp (optionnel)
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
SMS_PROVIDER=twilio
```

Étape 6 — Build et migrations

```
bash — build et migrations
```

```
cd /home/telepro/app

# Installer les dépendances
npm install

# Builder le frontend React !' apps/web/dist/
cd apps/web && npm run build && cd ../..

# Builder l'API TypeScript !' apps/api/dist/
cd apps/api && npm run build && cd ../..

# Appliquer toutes les migrations
export DATABASE_URL=postgresql://telepro:MotDePasse!@localhost:5432/telepro_ci

for f in 001 002 003 004_notifications 004_features 005 006 007 \
    008 009 010 011 012 013 014 015 016; do
    psql $DATABASE_URL -f apps/api/migrations/${f}*.sql
done
```

Étape 7 — Lancer avec PM2

```
bash — PM2
```

```
# Créer le répertoire de logs
```

```
sudo mkdir -p /var/log/telepro && sudo chown telepro:telepro /var/log/telepro
```

```
# Fichier de configuration PM2
```

```
cat > /home/telepro/app/apps/api/ecosystem.config.js << 'EOF'
```

```
module.exports = {  
  apps: [{  
    name:      'telepro-api',  
    script:    'dist/index.js',  
    instances: 2,  
    exec_mode: 'cluster',  
    env_file:  '.env',  
    error_file: '/var/log/telepro/error.log',  
    out_file:  '/var/log/telepro/out.log',  
    restart_delay: 3000,  
    max_restarts: 10,  
  }]  
};  
EOF
```

```
# Démarrer l'API
```

```
cd /home/telepro/app/apps/api && pm2 start ecosystem.config.js
```

```
# Activer le démarrage automatique au reboot
```

```
pm2 save && pm2 startup # '!' copier-coller la commande sudo affichée
```

Étape 8 — Nginx (proxy + frontend statique)

```
bash
```

```
sudo nano /etc/nginx/sites-available/telepro
```

/etc/nginx/sites-available/telepro

```
server {
    listen 80;
    server_name votre-domaine.ci www.votre-domaine.ci;

    # Frontend React (fichiers statiques)
    root /home/telepro/app/apps/web/dist;
    index index.html;

    # SPA routing – toutes les routes !' index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API – proxy vers Node.js port 3001
    location /api/ {
        proxy_pass          http://localhost:3001;
        proxy_set_header    Host $host;
        proxy_set_header     X-Real-IP $remote_addr;
        proxy_set_header     X-Forwarded-Proto $scheme;
        proxy_read_timeout  60s;
    }

    # Cache assets frontend (1 an)
    location ~* \.(js|css|png|jpg|svg|ico|woff2)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    gzip on;
    gzip_types text/plain text/css application/json application/javascript;
}
```

bash — activation Nginx + SSL

```
# Activer le site
sudo ln -s /etc/nginx/sites-available/telepro /etc/nginx/sites-enabled/
sudo nginx -t          # vérifier la syntaxe
sudo systemctl reload nginx

# Certificat SSL Let's Encrypt (HTTPS)
sudo certbot --nginx -d votre-domaine.ci -d www.votre-domaine.ci
# Certbot modifie Nginx automatiquement pour forcer HTTPS
```

3. Option B — Cloud managé sans administration serveur

Railway (API) + Neon (PostgreSQL) + Vercel (Frontend)

Cette option convient si vous ne voulez pas gérer un serveur Linux. Tous les services sont managés : mises à jour, sauvegardes et SSL sont automatiques. Coût total estimé : 10–15 \$/mois pour un trafic PME.

Architecture

Composant	Service	Plan gratuit	Coût estimé
Frontend React	Vercel	' Illimité	Gratuit
API Node.js	Railway	' 500h/mois	~5 \$/mois
PostgreSQL	Neon	' 500 Mo / 1 DB	Gratuit ou ~19 \$/mois
Domaine + DNS	O2switch	—	Existant

Étape 1 — PostgreSQL sur Neon (neon.tech)

```
bash — Neon PostgreSQL

# 1. Créer un compte sur https://neon.tech
# 2. New Project !' choisir région "Europe West" (Frankfurt)
# 3. Copier la DATABASE_URL fournie, ex :
#   postgresql://user:pass@ep-xxx.eu-central-1.aws.neon.tech/neondb?sslmode=require

# Appliquer les migrations depuis votre machine locale
export DATABASE_URL="postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require"

psql $DATABASE_URL -f apps/api/migrations/001_schema.sql
psql $DATABASE_URL -f apps/api/migrations/002_seed.sql
psql $DATABASE_URL -f apps/api/migrations/003_auth.sql
# ... jusqu'à 016_multi_magasins.sql
```

Étape 2 — API sur Railway (railway.app)

```
railway.app — configuration

# 1. Créer un compte sur https://railway.app
# 2. New Project !' Deploy from GitHub repo
# 3. Choisir le dépôt telepro-standard
# 4. Railway détecte Node.js automatiquement

# Dans Settings !' Service !' Root Directory : apps/api
# Build Command : npm install && npm run build
# Start Command : npm start

# Variables d'environnement à ajouter dans Railway Dashboard :
NODE_ENV=production
DATABASE_URL=postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require
JWT_SECRET=VOTRE_SECRET_64_CHARS
PORT=3001
```

Étape 3 — Frontend sur Vercel (vercel.com)

bash — Vercel

```
# Installer Vercel CLI
npm install -g vercel

# Depuis le dossier apps/web
cd apps/web

# Créer .env.production
echo "VITE_API_URL=https://votre-api.railway.app" > .env.production

# Déployer
vercel deploy --prod
# !' Vercel détecte Vite, build et déploie en ~2 minutes
# !' URL fournie : https://telepro-ci.vercel.app

# Ajouter votre domaine custom dans Vercel Dashboard
# puis pointer le CNAME chez 02switch vers cname.vercel-dns.com
```

Avantage Vercel + Railway : déploiement automatique à chaque git push sur main. Aucune intervention manuelle pour les mises à jour du code.

4. Option C + Script de déploiement automatisé

O2switch (front statique) + VPS Hetzner (API + DB)

Option C — Architecture mixte

Vous conservez O2switch pour le frontend et le domaine, et ajoutez un petit VPS (Hetzner CX11 à ~4 €/mois) uniquement pour l'API Express + PostgreSQL.

bash — Option C

```
# 1. Builder le frontend avec l'URL de l'API VPS
cd apps/web
echo "VITE_API_URL=https://api.votre-domaine.ci" > .env.production
npm run build

# 2. Uploader apps/web/dist/ vers O2switch via FTP ou rsync
rsync -avz apps/web/dist/ user@ftp.o2switch.net:~/public_html/

# 3. Pointer un sous-domaine api.votre-domaine.ci vers le VPS
#   Dans le manager O2switch : DNS Zone Editor
#   Ajouter un enregistrement A : api !' IP_VPS

# 4. Sur le VPS : installer uniquement l'API (étapes 1-7 de l'Option A)
#   Nginx sur le VPS n'héberge QUE /api/*
```

Script de mise à jour (toutes options VPS)

deploy.sh — à exécuter à chaque nouvelle version

bash — deploy.sh

```
#!/bin/bash
# /home/telepro/app/deploy.sh
set -e # stopper en cas d'erreur

echo "' Déploiement TéléPro CI $(date)"
cd /home/telepro/app

# 1. Récupérer le code
git pull origin main

# 2. Installer les nouvelles dépendances
npm install

# 3. Builder
npm run build

# 4. Appliquer les nouvelles migrations (décommenter si nécessaire)
# psql $DATABASE_URL -f apps/api/migrations/XXX.sql

# 5. Redémarrer l'API sans interruption (zero-downtime)
cd apps/api && pm2 reload ecosystem.config.js

echo "' Déploiement terminé"
```

```
bash
```

```
# Rendre le script exécutable
chmod +x /home/telepro/app/deploy.sh

# Utilisation
cd /home/telepro/app && ./deploy.sh
```

Vérifications post-déploiement

```
bash — vérifications
```

```
# Statut des processus
pm2 status
pm2 logs telepro-api --lines 30

# API répond ?
curl -s http://localhost:3001/api/auth/me | jq . # doit retourner 401

# Nginx OK ?
sudo nginx -t && sudo systemctl status nginx

# PostgreSQL OK ?
sudo systemctl status postgresql
psql $DATABASE_URL -c "SELECT COUNT(*) FROM utilisateurs;"

# Certificat SSL valide ?
curl -sI https://votre-domaine.ci | grep -i "HTTP\|strict"
```

5. Sécurité, sauvegardes et surveillance

Sécurité — Points essentiels

'JW
T_
SE
CR
ET
gén
éré
ave
c n
pm
run
gen
-se
cret
(mi
n
64
cha
rs, j
am
ais
par
déf
aut)
'Po
stgr
eS
QL
acc
essi
ble
uni
que
me
nt
en l
oca
l (p
ort
543
2 fe
rmé
en
ext
ern
e)
'AP
l po
rt 3
001
ina
cce
ssib
le d
irec
tem
ent
—
uni
que

me
nt
via
Ngi
nx
'HT
TP
S fo
rcé
par
Cer
tbot
(H
TT
P !' 3
01 r
edir
ect)
'Mo
t
de
pas
se
Pos
tgre
SQ
L
fort
(mi
n
20
cha
rs,
chif
fres
+
maj
usc
ule
s)
'Util
isat
eur
Lin
ux
non
-roo
t po
ur f
aire
tou
rne
r l'a
pp
&
Ne j
am
ais
co
mm
itter
le f
ichi
er .

env
da
ns
Git
Ré
gén
érer
JW
T_
SE
CR
ET
apr
ès t
out
e c
om
pro
mis
sio
n (r
évo
que
tou
tes
les
ses
sio
ns)

Sauvegarde automatique PostgreSQL

```
bash
```

```
# Créer le script de sauvegarde  
sudo nano /etc/cron.daily/telepro-backup
```

```
/etc/cron.daily/telepro-backup
```

```
#!/bin/bash  
BACKUP_DIR="/var/backups/telepro"  
DATE=$(date +%Y%m%d_%H%M)  
DB_URL="postgresql://telepro:MotDePasse!@localhost:5432/telepro_ci"  
  
mkdir -p $BACKUP_DIR  
  
# Dump compressé  
pg_dump -Fc $DB_URL > $BACKUP_DIR/telepro_$DATE.dump  
  
# Garder seulement les 14 derniers jours  
find $BACKUP_DIR -name "*.dump" -mtime +14 -delete  
  
# Optionnel : copier vers stockage externe (rclone, S3...)  
# rclone copy $BACKUP_DIR remote:backups-telepro  
  
echo "Backup OK : telepro_$DATE.dump"
```

bash — activation et test

```
sudo chmod +x /etc/cron.daily/telepro-backup
```

Tester manuellement

```
sudo /etc/cron.daily/telepro-backup
```

```
ls -lh /var/backups/telepro/
```

Restaurer une sauvegarde

```
pg_restore -Fc -d telepro_ci /var/backups/telepro/telepro_YYYYMMDD_HHMM.dump
```

Surveillance avec PM2 et alertes

bash — surveillance

Voir les logs en temps réel

```
pm2 logs telepro-api
```

Métriques CPU / mémoire

```
pm2 monit
```

Activer le monitoring PM2 Plus (gratuit jusqu'à 4 serveurs)

```
pm2 link CLE_SECRETE CLE_PUBLIQUE
```

Redémarrage automatique si l'API plante

(déjà configuré via max_restarts: 10 dans ecosystem.config.js)

Tester la résilience

```
pm2 kill # simule un crash
```

```
pm2 resurrect # ou le démarrage auto au reboot relancera
```

6. Récapitulatif et checklist de mise en production

Checklist Option A — VPS

'Se
rve
ur
VP
S U
bun
tu 2
2.0
4 c
om
ma
ndé
et
acc
essi
ble
en
SS
H
'Util
isat
eur
non
-roo
t cr
ée,
par
e-
feu
UF
W a
ctiv
é
'No
de.j
s
20,
Pos
tgre
SQ
L
16,
Ngi
nx,
PM
2 in
stall
és
'Ba
se
de
don
née
s te
lepr
o_c
i cr
éée
av
ec

utili
sat
eur
déd
ié
'Fic
hier
.en
v c
onfi
gur
é (DA
TA
BA
SE
_U
_RL,
JW
T_
SE
CR
ET)
'np
m i
nst
all
&&
np
m
run
buil
d e
xéc
uté
s s
ans
err
eur
'To
ute
s
les
mig
rati
ons
00
1!'01
6 a
ppli
qué
es
'PM
2 d
ém
arré
, p
m2
star
tup
con
figu
ré

'Ngi
nx
con
figu
ré
et r
ech
arg
é
'Ce
rtifi
cat
SS
L L
et's
En
cry
pt a
ctif
'DN
S
du
do
mai
ne
poi
nté
ver
s
l'IP
du
VP
S
'Te
st l
ogi
n
sur
http
s://
votr
e-d
om
ain
e.ci
'Scr
ipt
dep
loy.
sh
cré
é
et t
est
é
'Sa
uve
gar
de
cro
n c
onfi
gur
ée

Tableau récapitulatif des 3 options

Critère	Option A — VPS	Option B — Cloud	Option C — Mixte
Coût mensuel	4–8 €	10–15 \$	4–8 € + O2switch
Admin serveur	Requis	Aucune	Minimal
Déploiement initial	~3 heures	~30 minutes	~2 heures
Mise à jour code	./deploy.sh	git push	./deploy.sh
Sauvegarde DB	Manuel/cron	Automatique	Manuel/cron
SSL/HTTPS	Certbot	Automatique	Certbot (VPS)
Scalabilité	Limitée VPS	Automatique	Limitée VPS
Idéal pour	Contrôle max	Simplicité	Budget O2switch

Ports et accès réseau en production

Service	Port	Accessible depuis
Nginx HTTPS	443	Internet (public)
Nginx HTTP	80	Internet !' redirect 443
API Node.js	3001	Localhost uniquement (Nginx proxy)
PostgreSQL	5432	Localhost uniquement
SSH	22	Votre IP uniquement (recommandé)

Conseil : restreindre SSH à votre IP fixe pour éliminer les attaques par force brute.
sudo ufw delete allow OpenSSH && sudo ufw allow from VOTRE_IP to any port 22